Number-theoretic algorithms

Part VII Selected Topics

Divisibility, groups, and CRT support modular equations, powers, RSA, and randomized primality tests. A returned factor is a deterministic certificate. Pollard rho's time estimate needs an additional assumption. Prerequisites are divisibility, modular arithmetic, groups, and primes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover elementary number theory, greatest common divisors, modular arithmetic, modular linear equations, the Chinese remainder theorem, powers of an element, RSA, primality testing, and integer factorization.

The goals are to use Euclid and extended Euclid, solve modular equations and CRT systems, and distinguish RSA correctness, randomized testing guarantees, and factorization assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divisibility and input length

For integers d, a, $d \mid a$ means $a = kd$ for some integer k. A prime $p \geq 2$ has exactly the positive divisors 1 and p. The integer 1 is not prime. The gcd is nonnegative and positive unless both inputs are zero. Define $\gcd(0, 0) = 0$. Code first takes absolute values of negative inputs.

For $n \geq 1$, $a \equiv b \pmod{n}$ means $n \mid a - b$. The representative `a mod n` lies in $[0, n - 1]$, even for negative inputs. For example, $-7 \bmod 5 = 3$. Congruences preserve sums, differences, and products because their differences remain multiples of n. Division needs additional conditions.

A β-bit integer can have value near 2^β. A loop up to $\sqrt{n}$ is exponential in input length. We distinguish arithmetic operation counts from bit costs. Schoolbook multiplication and long-division remainder on $O(\beta)$-bit integers each take $O(\beta^2)$ bit operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bezout, cancellation, and factorization

For a, b not both zero, let g be the smallest positive integer of the form $ax + by$. Divide a by g. The remainder $a - qg$ is another integer linear combination. A positive remainder below g contradicts minimality, so $g \mid a$. Similarly $g \mid b$. Every common divisor divides g, proving $g = \gcd(a, b)$ and Bezout's identity $ax + by = g$.

If $\gcd(a, n) = 1$, write $au + nv = 1$. Multiplication by b gives $b = abu + nbv$, so $n \mid ab$ implies $n \mid b$. Coprimality permits cancellation. Modulo 6, $2 \cdot 1 \equiv 2 \cdot 4$, but $1 \not\equiv 4$.

If prime $p \nmid a$, then $\gcd(a, p) = 1$, so $p \mid ab$ implies $p \mid b$. Every $n \geq 2$ factors into primes by recursively factoring smaller divisors. Compare two factorizations. A prime in the first divides some prime in the second and therefore equals it. Cancel and induct to prove unique prime factorization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid's algorithm

For $a = qb + r$, $0 \leq r < b$, a common divisor of a, b divides $r = a - qb$. A common divisor of b, r divides $a = qb + r$. Thus $\gcd(a, b) = \gcd(b, r)$.

```
euclid(a,b):
    a=abs(a); b=abs(b)
    while b != 0: (a,b)=(b,a mod b)
    return a
```

The second argument strictly decreases to zero. The current gcd equals the original gcd, and the final $\gcd(a, 0) = a$ proves correctness. If $0 < b \leq a$, set $r = a \bmod b$. If $r \leq b/2$, the second argument already halves. Otherwise the next remainder is $b \bmod r = b - r < b/2$. Every two divisions halve it, giving $O(\log b)$ divisions, or $O(\beta)$ for β-bit inputs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci chains and bit bounds

If $a > b \geq 1$ takes k divisions to reach zero, tracing backward gives each earlier remainder at least the sum of the next two. With $F_1 = F_2 = 1$, this yields $a \geq F_{k+2}, b \geq F_{k+1}$. Fibonacci numbers at least double every two positions, so $k = O(\log b)$. Consecutive Fibonacci inputs have quotient 1 except at the final division and attain the same order of chain length.

The coarse per-division bound $O(\beta^2)$ times $O(\beta)$ rounds gives a safe $O(\beta^3)$ bit bound. This is not optimal. Tracking shrinking operands gives the sharper classical $O(\beta^2)$ gcd bound, but we do not use that refinement to claim measured speed.

Keeping two remainders uses $O(\beta)$ bits. Saving every trace row has additional storage cost. Euclid needs no known prime factorization.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Extended Euclid

If recursion on (b, r) returns $g = bx' + ry'$, substituting $r = a - qb$ gives $g = ay' + b(x' - qy')$. Return $(x, y) = (y', x' - qy')$. The base case $b = 0$ returns $(g, x, y) = (a, 1, 0)$.

```
extended(a,b):                  # a,b >= 0
    if b == 0: return (a,1,0)
    (g,x1,y1)=extended(b,a mod b)
    return (g,y1,x1-floor(a/b)*y1)
```

For 252 and 198, the divisions are $252 = 198 + 54$, $198 = 3 \cdot 54 + 36$, $54 = 36 + 18$, $36 = 2 \cdot 18$. Back substitution gives $18 = 54 - 36 = 4 \cdot 54 - 198 = 4 \cdot 252 - 5 \cdot 198$. The gcd is 18 and coefficients are $(4, -5)$. Adjust corresponding coefficient signs for negative original inputs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extended Euclid (continued)

Euclid: every remainder is an integer linear combination

Remainder r	252 coefficient x	198 coefficient y	Check 252x+198y
252	1	0	252+(0)=252
198	0	1	0+(198)=198
54	1	-1	252+(-198)=54
36	-3	4	-756+(792)=36
18	4	-5	1008+(-990)=18

Quotients 1,3,1,2. Last nonzero remainder: 18 = 4×252 − 5×198.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Modular groups and units

A group operation is closed and associative, has an identity, and gives every element an inverse. An abelian group also has a commutative operation. Inverses are unique. If b, c are inverses of a, then $b = b(ac) = (ba)c = c$.

The additive group $(\mathbb{Z}_n, +)$ is finite and abelian, with identity 0 and inverse $-a$. Multiplication on all of $\mathbb{Z}_n$ is generally not a group, since 0 has no inverse. For $n \geq 2$, the units $U(n) = \{a \in [0, n-1] : \gcd(a, n) = 1\}$ form the multiplicative group.

Bezout gives $au + nv = 1$ when $\gcd(a, n) = 1$, so u is an inverse. Conversely, if $au \equiv 1$, every common divisor divides $au - nv = 1$, so the gcd is 1. Products of units remain units because $(ab)^{-1} = b^{-1}a^{-1}$. Division by a has a unique modular meaning precisely when a is a unit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lagrange's theorem through cosets

A subgroup $H \subseteq G$ contains the identity and is closed under products and inverses. A left coset gH has $|H|$ elements because multiplication by g is a bijection. If $gh_1 = kh_2$, then $k^{-1}g = h_2h_1^{-1} \in H$, so $gH = kH$. Otherwise the cosets are disjoint. They partition finite G, proving $|H| \mid |G|$. A proper subgroup has size at most $|G|/2$.

A nonempty product-closed subset of a finite group is already a subgroup. For each a, finiteness gives $a^i = a^j$, $i < j$, so $a^{j-i} = e$ belongs to the subset. If $j - i > 1$, a^{j-i-1} is an inverse in the subset. If $j - i = 1$, then $a = e$, whose inverse is itself.

The order of a is the least positive t with $a^t = e$. The elements $a^0, \ldots, a^{t-1}$ are distinct and powers repeat modulo t, so its generated subgroup has size t. Lagrange gives $t \mid |G|$, hence $a^{|G|} = e$. Finiteness matters. The additive group of integers has elements of infinite order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Totients and distinct operations

Define $\varphi(n) = |U(n)|$. For prime p, the nonunits modulo p^α are exactly the $p^{\alpha-1}$ multiples of p. Hence $\varphi(p^\alpha) = p^{\alpha-1}(p-1)$. CRT below identifies units of a coprime product with the Cartesian product of component unit groups, giving $\varphi(n) = \prod_i p_i^{\alpha_i-1}(p_i-1)$.

For 15, $U(15) = \{1, 2, 4, 7, 8, 11, 13, 14\}$, so $\varphi(15) = 8$. Since $7 \cdot 13 \equiv 1$, the modular fraction $4/7$ means $4 \cdot 13 \equiv 7$, not integer quotient 0.

In additive $\mathbb{Z}_8$, 2 generates $\{0, 2, 4, 6\}$, with order 4. In multiplicative $U(7)$, 2 has powers 1,2,4 and order 3. The powers of 3 are 1,3,2,6,4,5 and its order is 6. The notation $\langle a \rangle$ depends on the operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All modular linear solutions

For $ax \equiv b \pmod{n}$, $n \geq 1$, set $g = \gcd(a, n)$. Necessarily $g \mid b$, since $ax + ny = b$. Extended Euclid gives $au + nv = g$. If $b = gh$, then $x_0 = uh$ is a solution, proving sufficiency.

Divide by g to get $(a/g)x \equiv b/g \pmod{n/g}$. Its coefficient is coprime to the new modulus, so it has one solution modulo n/g. Subtracting two solutions and cancelling gives $n/g \mid x - x_0$. All g distinct solutions modulo n are therefore

$$x = x_0 + k(n/g) \pmod{n}, \qquad k = 0, \ldots, g - 1.$$

This proves existence, number, completeness, and an algorithm. If $a = 0$, then $g = n$. Every representative solves it when $n \mid b$, and none do otherwise. For $n = 1$, the only representative is 0. We do not assert general unit division for that modulus.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equation example and output cost

For $18x \equiv 30 \pmod{42}$, $g = 6 = -2 \cdot 18 + 42$, so $x_0 = -10 \equiv 4 \pmod{7}$. The six solutions modulo 42 are 4,11,18,25,32,39. For example, $18 \cdot 4 \equiv 30$, and increasing x by 7 increases the product by $126 = 3 \cdot 42$. Replacing 30 with 31 gives no solution, since $6 \nmid 31$. The inverse of 18 modulo 42 does not exist.

The compressed triple $(x_0, n/g, g)$ avoids enumeration. Listing all solutions requires $\Omega(g)$ outputs. After normalizing $a \bmod n$, Euclid plus enumeration takes $O(\log n + g)$ arithmetic operations. Each output still has $O(\log n)$ bits, which must be counted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

Chinese remainder construction

For pairwise coprime $m_1, \ldots, m_k \geq 2$, let $M = \prod_i m_i$, $M_i = M/m_i$, $z_i = M_i^{-1} \pmod{m_i}$, $e_i = M_i z_i$, and $x = \sum_i a_i e_i \pmod{M}$.

For fixed j, every e_i with $i \neq j$ is zero modulo m_j, while $e_j \equiv 1$. Thus $x \equiv a_j \pmod{m_j}$, proving existence. If x, x' both solve the system, each m_i divides their difference. Pairwise coprimality and cancellation imply the product divides the difference, proving uniqueness modulo M.

The map $x \mapsto (x \bmod m_1, \ldots, x \bmod m_k)$ is a bijection preserving addition and multiplication. A residue is a unit exactly when every component is a unit. Otherwise it shares a prime factor with some component modulus. This proves multiplicativity of the totient for coprime products.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CRT selectors reconstruct 29

Solve $x \equiv 2 \pmod 3, x \equiv 4 \pmod 5, x \equiv 1 \pmod 7$. Here $M = 105$, $M_i = (35, 21, 15)$, inverses are $(2, 1, 1)$, and selectors are $(70, 21, 15)$.

$$x = 2 \cdot 70 + 4 \cdot 21 + 15 = 239 \equiv 29 \pmod{105}.$$

Indeed, 29 has residues 2,4,1. The selector coordinates are $(1, 0, 0), (0, 1, 0), (0, 0, 1)$. Modulo M, $e_i^2 = e_i$, $e_i e_j = 0$ for $i \neq j$, and $\sum_i e_i = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT selectors reconstruct 29 (continued)

CRT selectors: each e_i selects one component

$e_1 = 70$	$e_2 = 21$	$e_3 = 15$
mod (3,5,7)	mod (3,5,7)	mod (3,5,7)
(1,0,0)	(0,1,0)	(0,0,1)
$a_1 \cdot e_1 = 2 \times 70 = 140$	$a_2 \cdot e_2 = 4 \times 21 = 84$	$a_3 \cdot e_3 = 1 \times 15 = 15$

$$140 + 84 + 15 = 239 \equiv 29 \pmod{105}$$

29 → (2,4,1). Reconstruction and reduction are inverse maps.

Intermediate integers must not overflow. The bit length of M is of order $\sum_i \log m_i$. Use large integers and reduction as needed. The numerical value of M is not its memory cost in bits.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Noncoprime compatibility

For $x \equiv a \pmod{m}, x \equiv b \pmod{n}$, let $g = \gcd(m, n)$. Substituting $x = a + mk$ gives $mk \equiv b - a \pmod{n}$. A solution exists exactly when $g \mid b - a$, and its period is $\operatorname{lcm}(m, n) = mn/g$, not mn.

For $x \equiv 2 \pmod{6}, x \equiv 5 \pmod{9}$, the difference 3 is divisible by $g = 3$. The equation $6k \equiv 3 \pmod{9}$ becomes $2k \equiv 1 \pmod{3}$, so $k \equiv 2$, giving $x \equiv 14 \pmod{18}$. Replacing 5 by 4 gives no solution.

Merge general systems one congruence at a time, checking compatibility and updating the lcm. The pairwise-coprime selector formula cannot be applied directly to 6 and 9, because the inverses do not exist.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Powers, orders, and logarithms

For $a \in U(n)$, Lagrange gives Euler's theorem $a^{\varphi(n)} \equiv 1 \pmod{n}$. For prime p, this becomes Fermat's theorem $a^{p-1} \equiv 1$. Every residue satisfies $a^p \equiv a \pmod{p}$, with the zero case holding directly.

An element g of order $\varphi(n)$ is a primitive root and generates all units. Each unit equals g^k for a unique $k \pmod{\varphi(n)}$, its discrete logarithm $\operatorname{ind}_{n,g}(a)$. By the order definition, $g^x = g^y$ exactly when $x \equiv y \pmod{\varphi(n)}$.

Modulo 7, base 3 maps exponents 0,1,2,3,4,5 to 1,3,2,6,4,5, so $\operatorname{ind}(4) = 4$. Base 2 reaches only 1,2,4 and has no logarithm for 3. A nongenerator need not represent every unit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite fields have multiplicative generators

In a finite abelian group, elements of coprime orders r, s have a product of order rs. If $(ab)^k = 1$, then $a^k = b^{-k}$ lies in both generated subgroups. Their intersection has order dividing both r, s, so it contains only the identity. Thus $r \mid k, s \mid k$. Conversely the rsth power is 1.

Let L be the lcm of element orders in $\mathbb{F}_p^\times$. For each prime ℓ, some element order supplies the highest power ℓ^v in L. A suitable power of that element has order exactly ℓ^v. Multiplying the elements for distinct primes gives an element of order L.

All $p-1$ nonzero elements are roots of $X^L - 1$. A nonzero degree-L polynomial over a field has at most L roots, by repeated factorization, so $p - 1 \leq L$. Lagrange makes each order divide $p-1$, giving $L \leq p - 1$. Hence $L = p - 1$, and a generator exists. Arbitrary finite groups need not be cyclic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Generators for odd prime powers

For odd prime p, take a generator g modulo p. Modulo p^2,

$$(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p.$$

The second term divided by p is nonzero modulo p, so at least one $h \in \{g, g+p\}$ satisfies $h^{p-1} = 1 + cp$, $p \nmid c$.

If $v_p(z-1) = j \geq 1$, the binomial expansion gives $v_p(z^p - 1) = j + 1$. The first term is $p(z-1)$. All later terms have at least one more factor of p, including the last because $p \geq 3$, so they cannot cancel the first. Thus $v_p(h^{(p-1)p^k} - 1) = k + 1$.

Reduction modulo p makes the order modulo p^α divisible by $p-1$. Lagrange makes it divide $p^{\alpha-1}(p-1)$. The valuation excludes every smaller power of p, so the order equals the totient. Hence $U(p^\alpha)$ is cyclic. Reusing a generator modulo p unchanged requires the p^2 check.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exactly which unit groups are cyclic

For $n > 1$, the possibilities are exactly $2, 4, p^\alpha, 2p^\alpha$, with p odd prime. The previous proof covers odd prime powers. CRT gives $U(2p^\alpha) \cong U(2) \times U(p^\alpha)$, whose first component is trivial. The cases 2 and 4 are direct.

If $8 \mid n$, reduction maps $U(n)$ onto $U(8)$. Every element of $U(8)$ squares to 1, but the group has four elements and is not cyclic. A cyclic group's homomorphic image must be cyclic, a contradiction. Surjectivity follows by choosing an odd residue in the power-of-two component and unit residues in the odd prime-power components via CRT.

If n has two distinct odd prime factors, independent CRT signs give at least four units squaring to 1. If $4 \mid n$ and an odd prime divides n, its two components also give four roots. A cyclic group has at most two solutions to $g^{2k} = 1$ modulo its order. These cases exclude all remaining forms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nontrivial square roots expose factors

For an odd prime power p^α, $x^2 \equiv 1$ implies $p^\alpha \mid (x-1)(x+1)$. The gcd of the two factors divides 2, so the odd prime p cannot divide both. One factor contains all of p^α, giving $x \equiv \pm 1$.

If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv \pm 1$, then n is composite and both $\gcd(x-1, n)$ and $\gcd(x+1, n)$ are proper nontrivial factors. Neither equals n, or x would equal a sign. Neither equals 1, since cancellation would force the other factor to be divisible by n.

For $n = 77$, $x = 34$, $34^2 = 1156 \equiv 1$, yet $34 \neq 1, 76$. The gcds are 11 and 7. CRT interprets 34 as +1 modulo 11 and -1 modulo 7, with unsynchronized signs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Square and multiply

For $n \geq 1, b \geq 0$, normalize $a \bmod n$ and initialize $r = 1 \bmod n$. Read every exponent bit from highest to lowest.

```
r = 1 mod n
for bit in binary(b), highest first:     # empty string for b=0
    r = r*r mod n
    if bit == 1: r = r*a mod n
return r
```

After reading prefix v, the invariant is $r \equiv a^v \pmod{n}$. The next bit δ changes it to $2v + \delta$, exactly the effect of squaring and optionally multiplying by a. The empty prefix has value 0. The final prefix is b, proving correctness. Exponent 0 returns $1 \bmod n$, including 0 when $n = 1$. Coprimality is unnecessary. Negative exponents need an inverse and a separate input contract.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

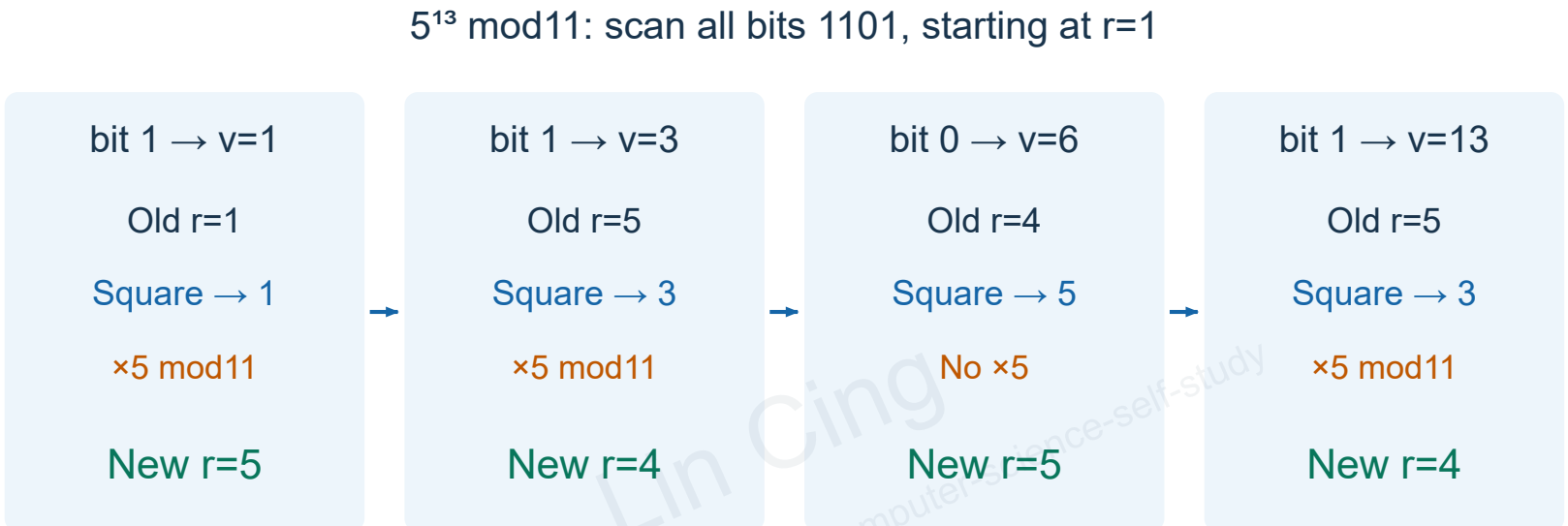
Every bit of 5 to the thirteenth

The binary representation of 13 is 1101. Starting with $r = 1$, scan all four bits modulo 11.

Bit	New prefix	Old r	Square modulo 11	Multiply by 5 or retain
1	1	1	1	5
1	3	5	3	4
0	6	4	5	5
1	13	5	3	4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every bit of 5 to the thirteenth (continued)



Initializing $r = a$ means the leading 1 was already read, so only 101 remains. Starting at a but scanning all of 1101 instead follows exponents 3,7,14,29 and values 4,3,9,9, incorrectly returning 9. A k-bit exponent uses at most $2k$ modular multiplications. With a β-bit modulus, ordinary bit cost is $O(k\beta^2)$, or $O(\beta^3)$ when $k = O(\beta)$. Initial reduction of an exceptionally long base has additional cost.

RSA on every residue

Choose distinct odd primes p, q, set $n = pq$, $\varphi = (p-1)(q-1)$, and choose $1 < e < \varphi$ with $\gcd(e, \varphi) = 1$. Extended Euclid gives $d = e^{-1} \pmod{\varphi}$, $1 \le d < \varphi$. Publish (n, e), retain d, p, q. For $0 \le m < n$, encrypt $c = m^e \bmod n$ and decrypt $m' = c^d \bmod n$.

Write $ed = 1 + k(p-1)(q-1)$. For any m, modulo p, if $p \mid m$, both m^{ed} and m are zero. Otherwise Fermat gives $m^{p-1} = 1$, so $m^{ed} = m$. The same applies modulo q. CRT uniqueness proves $m^{ed} \equiv m \pmod{n}$, including nonunits. Euler's theorem alone would omit those legal messages.

The private and public powers also undo each other in the opposite order, so they are inverse mathematical permutations. This correctness is separate from protocol security.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A nonunit RSA message

Take $p = 11, q = 17, n = 187, \varphi = 160, e = 7$. Since $1 = 23 \cdot 7 - 160$, $d = 23$. The message $m = 11$ is a legal nonunit.

Encryption uses $11^2 \equiv 121, 11^4 \equiv 55$, and $11^7 \equiv 55 \cdot 121 \cdot 11 \equiv 88 \pmod{187}$. Decryption uses $88^2 \equiv 77, 88^4 \equiv 132, 88^8 \equiv 33, 88^{16} \equiv 154$. Since $23 = 16 + 4 + 2 + 1$, multiply the corresponding values. The reductions are $154 \cdot 132 \equiv 132$, $132 \cdot 77 \equiv 66$, and $66 \cdot 88 \equiv 11$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A nonunit RSA message (continued)

n=187, e=7, d=23. Message m=11 is not a unit.



mod11 component	mod17 component
$11 \to 0$; $88 \to 0$; $0^{23} \to 0$	$11 \to 11$; $88 \to 3$; $3^{23} \to 11$
p | m: directly recover 0	Unit component: apply Fermat

CRT: (0 mod11, 11 mod17) uniquely reconstructs 11 mod187

A fixed-bit-length public exponent costs $O(\beta^2)$ with ordinary arithmetic. A private exponent of $O(\beta)$ bits costs $O(\beta^3)$. Factoring n reveals p, q, hence φ, d. This does not prove the converse claim that RSA inversion is equivalent to factoring.

Algebraic limits of textbook RSA

Without encoding, RSA is deterministic. Equal messages give equal ciphertexts, so a small message space can be enumerated. Also $E(m_1)E(m_2) \equiv E(m_1 m_2)$. Given $c = E(m)$ and $r \in U(n)$, the ciphertext cr^e represents mr. This is malleability, not a security proof.

Using m^d directly as a signature also lacks a complete security construction. Choose any σ, then set $m = \sigma^e$. This creates a pair that passes verification without an authorized signature on a meaningful message. Practical schemes separately specify randomized encodings, hashes, and verification formats. This chapter derives the permutation rather than deployment parameters.

These algebraic weaknesses can coexist with hard factorization of large integers. Decryption correctness cannot replace an attack model and security definition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Primality tests

Trial division needs divisors only through $\lfloor \sqrt{n} \rfloor$. If $n = ab$ and both factors exceed $\sqrt{n}$, their product exceeds n. Absence of a small factor therefore proves primality. For β-bit n, $\sqrt{n}$ is about $2^{\beta/2}$, not polynomial in bit length.

The prime number theorem gives $\pi(X) \sim X / \ln X$. Uniform sampling in a comparable-magnitude interval has approximate prime density $1/\ln X$, or $2/\ln X$ among odd candidates. This asymptotic estimate does not guarantee a prime within $\ln X$ trials or prove the test's error bound.

A Fermat test finding $a^{n-1} \not\equiv 1$ proves compositeness. Equality gives only a probable prime. For $341 = 11 \cdot 31$, $2^{10} = 1024 \equiv 1$, so $2^{340} = 1$ despite compositeness. Carmichael numbers let every unit pass. For $561 = 3 \cdot 11 \cdot 17$, all of 2,10,16 divide 560, so Fermat and CRT give $a^{560} = 1$ for every unit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Miller-Rabin boundaries and square chains

Handle $n < 2$ as nonprime, $n = 2, 3$ as prime, and all other even n as composite. For odd $n \geq 5$, write $n - 1 = 2^s d$, d odd, $s \geq 1$. Choose a uniformly among integers in $[2, n - 2]$.

```
x = modular_power(a,d,n)
if x == 1 or x == n-1: pass this round
else:
    repeat s-1 times:
        x = x*x mod n
        if x == n-1: pass this round
    if no -1 occurred: return COMPOSITE
```

Repeat with independent bases and return probably prime only if every round passes. For prime n, Fermat gives $a^{n-1} = 1$. If the chain does not start at 1, the value before its first 1 must be -1, since a field has only square roots ± 1 of 1. Thus primes always pass, and every rejection proves compositeness. Without the special case for 3, sampling would use the empty interval $[2, 1]$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

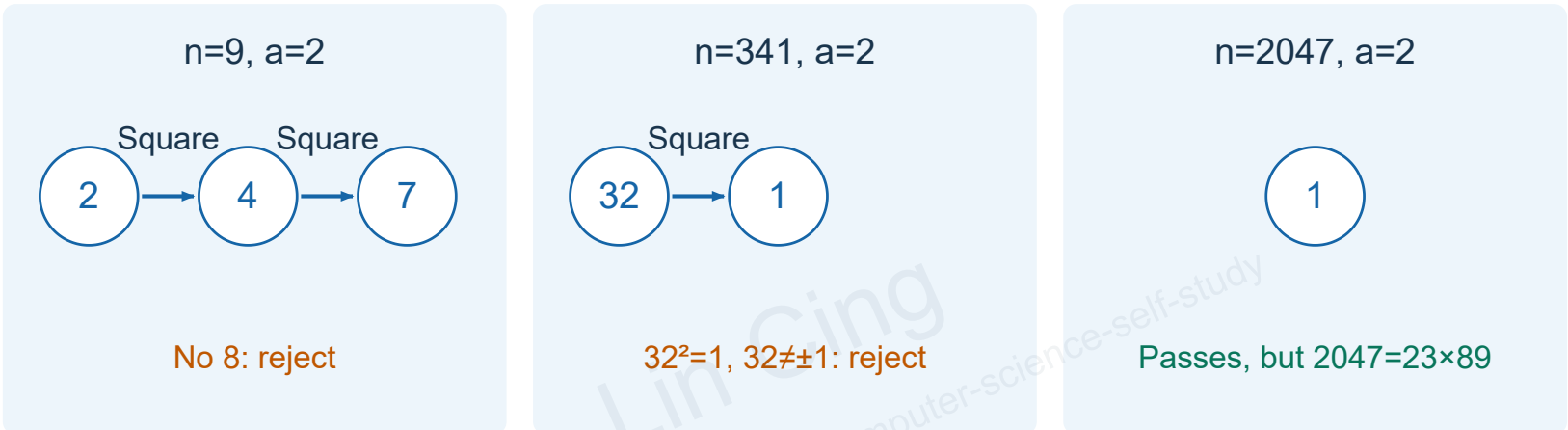
Strong testing separates Fermat examples

n, a	Decomposition	Initial power	Later squares	Outcome
5,2	$4 = 2^2 \cdot 1$	2	$4 = -1$	Pass
9,2	$8 = 2^3 \cdot 1$	2	4,7	Reject, no -1
341,2	$340 = 2^2 \cdot 85$	32	1	Reject, nontrivial root 32
2047,2	$2046 = 2 \cdot 1023$	1	None needed	Pass, but $2047 = 23 \cdot 89$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strong testing separates Fermat examples (continued)

Miller-Rabin: interpret −1 within the same chain



Passing means probable prime. Repeating a fixed base does not amplify confidence.

A fixed single base cannot prove primality for general integers. The error is acceptance of a composite. Primes are never rejected. The proof below bounds random-base acceptance for each fixed composite.

Count roots in prime-power components

Fix odd composite $n = \prod_{i=1}^{r} p_i^{\alpha_i}$, with $n - 1 = 2^s d$. Write $p_i - 1 = 2^{t_i} u_i$, with odd u_i. Let $D_i = \gcd(d, u_i)$, $\tau = \min(s, t_1, \ldots, t_r)$. A passing base must be a unit. If $p_i \mid a$, every positive power in that component is zero, never either sign.

The group $\mathbb{F}_{p_i}^{\times}$ has a generator. Write $a = g^k$. The equation $a^d = 1$ has D_i roots. The equation $a^{2^j d} = -1$ becomes a linear congruence in k. The coefficient gcd is $2^{\min(j,t_i)} D_i$, which divides $(p_i - 1)/2$ exactly when $j < t_i$. In that case there are $2^j D_i$ roots.

Each root lifts uniquely to $p_i^{\alpha_i}$. If z solves $X^h - \eta = 0$ modulo p_i^a, try $z + c p_i^a$. At the next level solve

$$(z^h - \eta)/p_i^a + h z^{h-1} c \equiv 0 \pmod{p_i}.$$

For $h = 2^j d$, $p_i \nmid h$ because $p_i \nmid n - 1$. Also z is a unit. The derivative is nonzero, so c is unique. The root counts therefore hold at every prime-power component.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT counts synchronized strong liars

Passing cases are disjoint. Either $a^d = 1$, or a unique $j \in [0, s-1]$ satisfies $a^{2^j d} = -1$. Two levels cannot both be -1, because squaring the first gives 1 thereafter. CRT makes components independent, but every component must reach -1 at the **same level**.

Thus the exact strong-liar count in $[1, n-1]$ is

$$L = \left(1 + \sum_{j=0}^{\tau-1} 2^{jr}\right) \prod_i D_i.$$

The first term counts initial 1 in every component. The term for j counts synchronized -1 at that level. Combining components that reach -1 at different levels overcounts bases that the test rejects. Factorization is used for analysis. The algorithm does not need the factors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One-quarter bound with one or at least three factors

If $r = 1$, then $n = p^\alpha, \alpha \geq 2$. Since $p - 1 \mid p^\alpha - 1, \tau = t, D = u$, so $L = p - 1$. Therefore

$$\frac{L}{n-1} = \frac{p-1}{p^\alpha - 1} \leq \frac{1}{p+1} \leq \frac{1}{4}.$$

If $r \geq 3$, use $\varphi(n) = \prod_i p_i^{\alpha_i - 1} 2^{t_i} u_i$, $D_i \leq u_i$, and $t_i \geq \tau$ to get

$$\frac{L}{\varphi(n)} \leq \frac{1 + \sum_{j=0}^{\tau-1} 2^{jr}}{2^{r\tau}} = \frac{1}{2^r - 1} + \frac{2^r - 2}{(2^r - 1)2^{r\tau}}.$$

This decreases for $\tau \geq 1$. Its maximum at 1 is $2^{1-r} \leq 1/4$. Since $\varphi(n) \leq n - 1$, $L \leq (n-1)/4$. For $n = 9$, $L = 2 = (n-1)/4$, so this constant cannot be lowered on the full representative domain.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The necessary two-factor case

For $r = 2$, if $t_1 \neq t_2$ or $s < \min(t_1, t_2)$, then $t_1 + t_2 \geq 2\tau + 1$. Thus

$$\frac{L}{\varphi(n)} \leq \frac{1 + \sum_{j=0}^{\tau-1} 4^j}{2^{2\tau+1}} = \frac{4^\tau + 2}{6 \cdot 4^\tau} \leq \frac{1}{4}.$$

Otherwise $t_1 = t_2 = \tau \leq s$. If some $\alpha_i \geq 2$, the extra prime-power factor in the totient is at least 3. If some $D_i < u_i$, the odd integer u_i / D_i is at least 3. At least one occurs. Otherwise $n = pq$ is squarefree and $D_i = u_i$, so both $p - 1, q - 1$ divide $n - 1$. Reducing $pq - 1$ modulo $p - 1$ gives $p - 1 \mid q - 1$. The reverse gives $q - 1 \mid p - 1$, forcing $p = q$, a contradiction.

The extra divisor of at least 3 gives $L/\varphi(n) \leq (4^\tau + 2)/(9 \cdot 4^\tau) \leq 1/6 < 1/4$. Every odd composite case is covered without an unspecified subgroup or Carmichael exception.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sampling and posterior probability

Bases 1 and $n-1$ always pass. Removing them from $L \le (n-1)/4$ gives $(L-2)/(n-3) \le 1/4$ on $[2, n-2]$. For each **fixed composite** n, R independent uniform rounds therefore accept with probability at most 4^{-R}. This is not directly the probability that an accepted candidate is composite.

If the candidate distribution has prior prime probability $\pi > 0$, prime pass probability is 1 and composite acceptance is at most $\epsilon = 4^{-R}$. Bayes gives

$$\Pr(\text{composite} \mid \text{pass}) \le \frac{\epsilon(1-\pi)}{\pi + \epsilon(1-\pi)}.$$

The candidate distribution must supply π. An adversarial fixed input cannot be assigned a uniform prime density without evidence.

Each round uses $O(\log n)$ modular multiplications and ordinary bit cost $O(\beta^3)$. R rounds cost $O(R\beta^3)$, plus sampling. A fixed base list is deterministic only on a proved input range. Reusing one randomly selected base does not give independence. Independent sampling with replacement may happen to repeat a value and still satisfies the contract.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Factorization and Pollard rho

Proving compositeness need not reveal factors. Trial division finds them deterministically but can require $\sqrt{n}$ trials. Pollard rho seeks a collision modulo an unknown factor.

For $f(x) = x^2 + c \bmod n$ and $p \mid n$, reducing $f(x)$ modulo p equals applying the same polynomial to $x \bmod p$. If $x_i \equiv x_j \pmod{p}$, then $p \mid x_i - x_j$, so $d = \gcd(|x_i - x_j|, n) > 1$. If $x_i \not\equiv x_j \pmod{n}$, then $d < n$, yielding a proper factor.

This implication is deterministic. Verify a returned value by $1 < d < n, d \mid n$. The uncertain parts are the collision time and whether all components collide together. The latter gives $d = n$, which is not successful factorization.

Lin Cing https://github.com/xinhi6716/computer-science-self-study/

Floyd and checkpoint schedules

```
x=y=x0
repeat until work budget exhausted:
    x=f(x); y=f(f(y))
    d=gcd(abs(x-y),n)
    if 1<d<n: return d
    if d==n: restart with another x0 or c
return no factor found within budget
```

Floyd advances the slow pointer by one and the fast pointer by two. A finite functional graph has tail length μ and cycle length λ. Once both are on the cycle, their relative position changes by one per round, so they meet within λ more rounds. Cost is $O(\mu + \lambda)$ function evaluations and a constant number of stored integers.

Another schedule fixes a checkpoint $y = x_{2^k}$ while x advances to the next power of two. Once the checkpoint is on the cycle and the interval is at least λ, the interval contains a matching component. This is a different schedule, so its trace must not be mixed with Floyd's. Both need a budget and a rule for $d = n$.

Three rounds factor 8051

Use $n = 8051$, $x_0 = 2$, $f(x) = x^2 + 1 \bmod 8051$, with Floyd's schedule.

Round	x	y	Absolute difference	gcd with 8051
1	5	26	21	1
2	26	7474	7448	1
3	677	871	194	97

In round 3, 677 and 871 both have residue 95 modulo 97, while their residues modulo 83 are 13 and 41. The collision yields 97 rather than 8051. Division gives 83. Trial division through their square roots confirms both are prime, completing $8051 = 83 \cdot 97$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Three rounds factor 8051 (continued)

Floyd round 3: x=677, y=871, difference 194

mod97: collision

677 mod97	871 mod97
95	95

mod83: no collision yet

677 mod83	871 mod83
13	41

97 | 194, but 83 ∤ 194, so gcd(194,8051)=97

Check 8051 = 83×97. Success requires a proper divisor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Birthday estimates and their assumptions

If iteration modulo unknown prime factor p behaves approximately as a uniform random mapping, it explores new points until the first collision. The probability of no collision in k steps is approximately

$$\prod_{j=0}^{k-1}(1 - j/p) \le \exp(-k(k-1)/(2p)).$$

Typical collision scale is $\sqrt{p}$, and tail-cycle methods detect it in the same order of steps. A fixed polynomial $x^2 + c$ is not independent randomness. This assumption gives neither a worst-case time for that polynomial nor a success probability for each restart.

For balanced semiprimes, p is about $\sqrt{n}$, giving heuristic scale $n^{1/4} = 2^{\beta/4}$, still not polynomial in β. Every round also pays for a β-bit modular square and gcd. Using this chapter's safe coarse gcd bound gives $O(\beta^3)$, rather than constant cost.

Birthday estimates and their assumptions (continued)

After finding a factor, recursively factor both sides. Remove even factors first and specify the confidence of prime or probable-prime leaves. Exhausting a budget means no factor was found within it, not that the input is prime. The factor certificate is certain. Completion-time estimates remain conditional on the heuristic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 31.

M. O. Rabin, "Probabilistic Algorithm for Testing Primality," *Journal of Number Theory* 12(1), 1980, doi:10.1016/0022-314X(80)90084-0.

L. Monier, "Evaluation and Comparison of Two Efficient Probabilistic Primality Testing Algorithms," *Theoretical Computer Science* 12(1), 1980, doi:10.1016/0304-3975(80)90007-9.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study